

RFP Dev Questions handbook

Quote-ready answers to RFP questions about our build process, accessibility, security, forms, content management, and migration — for typical marketing/informational sites where the only end-user interactivity is forms.

RFPs often quote standards they find on google or from an AI written for the maximum case (logins, payments, stored PII). For a marketing site the honest answer isn't to claim all of it — it's to explain which standards apply and which exist for problems we don't have. The quote-ready answers below do that without sounding defensive.

How to use this:

1. Confirm the project fits **What "the typical project" means** below.
2. Scan **Talk to Tucker if**.
3. If neither throw up any flags, freely paste from each section's **quote-ready answer**; use the **plain-English version** to understand deeper or for follow-up.

What "the typical project" means

Unless we say otherwise, this handbook describes a project where:

- The site is **informational or marketing** — homepage, services, case studies, blog, contact.
- The only **end-user interactivity is forms** going to email or Netlify Forms; no database.
- **No logins, no payments, no personal-data storage** beyond form submissions.
- **Editors are trusted client staff** working in Prismic (which handles their auth, roles, and version history).
- **Fewer than ~1,000 pages.**

Talk to Tucker if

The site does more than marketing

- Multi-step workflows — grant applications, volunteer onboarding, configurators
- Complex interactive components — calculators, data visualizations

Data handling and privacy

- Forms collect data someone wouldn't want leaked (donor records, job applications, health intake)
- Full consent management platform required (Cookiebot, OneTrust)
- GDPR-bound clients with marketing-automation integration

Higher accessibility bar

- WCAG AAA, VPAT/ACR, third-party audit, or Section 508 trusted-tester process
- EU public-sector client (Web Accessibility Directive)

Vendor / hosting compliance

- RFP requires the **vendor** (us, not Prismic / Netlify) to hold SOC 2 or ISO 27001
- Pre-launch penetration testing or bug-bounty program in scope
- FedRAMP / StateRAMP authorized hosting

Migration scale

- Over ~1,000 pages or multiple source systems
- Institutional archive with records-retention requirements

Technical stack and build process

The development side of the engagement — stack, why it was chosen, and how an approved design becomes a working site.

QUOTE-READY ANSWER

The site is built on **SvelteKit** (application framework), **Tailwind CSS** (styling), **Prismic** (the CMS editors use after launch), and **Netlify** (hosting and continuous deployment). This stack is chosen for content-driven websites: SvelteKit ships minimal JavaScript so pages load fast on cellular, Prismic is structured around reusable content blocks editors compose without developer involvement, and Netlify produces a live preview URL on every code change so stakeholders review real builds, not mockups. Pages are composed from a library of reusable content blocks called "slices" — typed, designed patterns editors can drop into any page — so most pages are recombinations of existing blocks rather than new layouts.

PLAIN-ENGLISH VERSION

Why this stack:

Tool	Why this choice
SvelteKit	Compiles to minimal JavaScript instead of shipping a heavy framework runtime. Fast loads on cellular and slow connections — critical for content-driven sites where most traffic is first-time visitors.
Tailwind CSS	Utility-first styling enforces design-system consistency by construction; no separate stylesheet to drift over time. One-to-one mapping between Figma design tokens and code.
Prismic	Content models are defined in code and version-controlled, not in a vendor UI. Editors fill in structured fields, not HTML — they can't break the design. SOC 2 Type 2.
Netlify	Git-based deployment with a live preview URL on every code change. Handles HTTPS, redirects, form submissions, and DDoS protection at the edge — no separate infrastructure to maintain. SOC 2 Type 2.

Why slices:

A content-driven site is mostly variations on a small set of patterns: heros, content sections, image grids, CTAs, testimonials, contact blocks. Building those patterns once as slices, rather than reimplementing them per page, has compound benefits:

- **Editors compose pages without a developer.** Adding or rearranging a landing page is editor work, not a code change.

- **The design system can't drift.** A "hero" looks like a hero everywhere; editors can't accidentally invent a new pattern that breaks from the brand.
- **Pages reuse, don't repeat.** Cost lives in the slice library, not in every page — most additional pages are recombinations of existing blocks, not new layouts.

Build process:

1. Design team delivers a Figma homepage signed off by the client.
2. Homepage built as Prismic slices, deployed to a preview URL.
3. Homepage iterated to client approval while remaining page designs are produced in parallel.
4. Remaining pages built against the same slice library.
5. Launch: client-provided redirects applied, DNS cutover.

Accessibility

QUOTE-READY ANSWER

We design and build to **WCAG 2.2 Level AA**, which is the standard referenced by Section 508, the standards typically applied for ADA web accessibility, and EN 301 549. Every code change is automatically tested against the axe-core ruleset for WCAG 2.0 / 2.1 / 2.2 A and AA violations, and Lighthouse CI gates accessibility scoring at 95/100 or above. Issues that surface beyond automated coverage are remediated when identified.

PLAIN-ENGLISH VERSION

WCAG 2.2 Level AA is the legal standard almost every public-sector and non-profit RFP requires. It's what we hit by default. In practice:

- Brand colors are checked for contrast (4.5:1 for normal text) before any code is written
- Image alt-text fields are present on every Prismic image; editors are trained to fill them in
- Forms have visible labels and errors that screen readers announce
- Component primitives are keyboard-navigable by default
- "Reduce motion" OS settings disable animations

Two common a11y issues on content-driven sites — low-contrast brand colors and unlabeled form fields — are addressed at the system level so they don't depend on individual care. Image alt text relies on editor diligence; we support that with training but can't enforce it at the CMS level (Prismic doesn't expose required-field validation by design).

Security

QUOTE-READY ANSWER

The site applies industry-standard defenses for content-driven websites: a strict Content Security Policy with explicit allowlists, HSTS preload, modern security headers (X-Frame-Options, X-Content-Type-Options, Referrer-Policy, Permissions-Policy, COOP), and automated dependency vulnerability scanning that fails the build pipeline on any high-severity finding (clients on a maintenance retainer also receive monthly grouped dependency-update PRs). The OWASP Top 10 risks that apply in this architecture — primarily A05 (Misconfiguration) and A06 (Vulnerable Components) — are gated in CI on every code change. The site has no user accounts, no database, and processes no payments, eliminating the majority of OWASP Top 10 categories by architecture rather than mitigation.

PLAIN-ENGLISH VERSION

The OWASP Top 10 is a list of common web application security risks. RFPs often quote it in full, but for a site of this architecture most items describe risks that don't physically exist:

OWASP risk	What it covers	Applies here?
A01 Broken Access Control	A user reaches data they shouldn't	No — no user accounts
A02 Cryptographic Failures	Encryption done badly	Only HTTPS, which we enforce
A03 Injection	Bad input damages a database	No database
A04 Insecure Design	Architecture has a flaw	Reviewed during discovery
A05 Security Misconfiguration	Headers/permissions wrong	Yes — this is what we focus on
A06 Vulnerable Components	A dependency has a known bug	Yes — scanned on every change
A07 Authentication Failures	Login system can be tricked	No login system
A08 Software Integrity Failures	Code or updates tampered with	Lockfile + signed deploys
A09 Logging & Monitoring	Can't see what happened	Server logs + CSP violation reports
A10 SSRF	Site fetches things for an attacker	No outbound URL fetching

Forms and visitor privacy

QUOTE-READY ANSWER

Contact forms use Netlify Forms (or similar). Submissions are protected in transit by TLS, stored encrypted at rest by the form provider, forwarded to designated client email addresses, and retained only as long as the client needs them. Spam is mitigated with honeypot fields and, where appropriate, hCaptcha. By default the site collects no analytics-grade visitor identifiers; when site analytics are required we recommend Plausible (privacy-respecting, no cookie banner needed) over Google Analytics, and when clients prefer GA we implement a basic cookie-consent banner alongside it.

PLAIN-ENGLISH VERSION

The only visitor data we touch is what someone voluntarily types into a form. We don't track, profile, fingerprint, or sell visitor information. For most projects, "data privacy" comes down to three things:

1. Form data is sent over HTTPS (default).
2. Data isn't retained longer than the client needs it.
3. We don't add tracking that triggers a consent banner unless the client wants it.

Many clients ultimately want Google Analytics or similar. When tracking is added, we implement a basic cookie-consent banner alongside it.

Content management

QUOTE-READY ANSWER

Content is managed in **Prismic**, a headless CMS where editors log in separately from the live site. Content has a defined structure — typed fields and reusable slices, not free-form HTML — so editors can author and rearrange pages without risking the design. Every change is versioned with full history; editors can schedule releases and preview drafts before they go live.

PLAIN-ENGLISH VERSION

- **Every change is versioned and reversible** — Prismic keeps history.
- **Editors can work ahead** — schedule a press release for next Tuesday, preview a page before it's live.
- **No security exposure from editor mistakes** — Prismic isn't where the public site lives.

Migration

QUOTE-READY ANSWER

When migrating from an existing site, the client provides the content to be carried over in a workable format (CMS export, URL list, document, spreadsheet — whatever's available). We model it in Prismic, import it, and the new site launches with all migrated content already in place.

PLAIN-ENGLISH VERSION

The client hands us the content they want migrated, in whatever format they have. We translate it into Prismic's content model — shaping it to fit the new design's slice library — and import it. The migrated content is already in Prismic before launch, so editors can review and adjust ahead of go-live.

Quick reference: short answers to common RFP questions

Q: Is the site WCAG 2.1 / 2.2 AA compliant? The site is designed and built to WCAG 2.2 AA standards, with automated axe-core testing on every code change and Lighthouse CI gating accessibility at 95/100. Issues beyond automated coverage are remediated when identified. Formal WCAG conformance attestation via third-party audit is out of default scope.

Q: Does the site comply with Section 508? Section 508 references WCAG 2.0 AA; we target WCAG 2.2 AA, which exceeds it. Formal Section 508 conformance attestation (VPAT/ACR, trusted-tester process) is out of default scope.

Q: Is the site SOC 2 / ISO 27001 compliant? SOC 2 and ISO 27001 are organizational certifications, not website features. Prismic and Netlify both carry SOC 2 Type 2 attestation. Vendor-level certification (us, separate from the platforms) is out of default scope.

Q: Do you support GDPR / CCPA / state privacy laws? The site collects no visitor data by default beyond what people type into forms. When clients add Google Analytics or similar tracking, we implement a basic cookie-consent banner alongside it. Privacy-policy authoring and granular consent management platforms (Cookiebot, OneTrust) remain out of default scope.